

第一题 图鉴收藏家的计算

【问题描述】

小 R 是一位魔法图鉴收藏家，最近从朋友小 W 那里得到了一本旧的魔法图鉴册。这本图鉴册中记录了各种魔法生物，每种魔法生物都有两个关键属性：栖息地和元素属性。已知栖息地共有 5 种：森林、沙漠、海洋、山脉、沼泽；元素属性共有 8 种：火、水、风、土、雷、冰、光、暗。

一本完整的魔法图鉴需要包含所有可能的组合：即每种栖息地与每种元素属性都恰好对应一种魔法生物。因此，一本完整的图鉴共有 $5 \times 8 = 40$ 种不同的魔法生物。

小 R 发现这本旧图鉴册可能不完整，于是他打算向魔法商店购买一些新的图鉴页。魔法商店提供所有种类的图鉴页，且数量充足。小 R 想知道，他至少需要购买多少张新的图鉴页，才能用现有的图鉴册和购买的图鉴页组成一本完整的魔法图鉴。

为了方便输入，我们使用以下编码：

栖息地：F（森林）、D（沙漠）、S（海洋）、M（山脉）、W（沼泽）

元素属性：F（火）、W（水）、A（风）、E（土）、T（雷）、I（冰）、L（光）、K（暗）

每张图鉴页用一个长度为 2 的字符串表示，第一个字符表示栖息地，第二个字符表示元素属性。例如："FW" 表示森林-水属性的魔法生物，"MT" 表示山脉-雷属性的魔法生物。

【输入描述】

输入的第一行包含一个整数 n ，表示现有图鉴页的数量。

接下来 n 行，每行包含一个长度为 2 的字符串，描述一张现有的图鉴页。

【输出描述】

输出一行一个整数，表示最少还需要购买多少张图鉴页

【输入样例 1】

```
1
MF
```

【输出样例 1】

```
39
```

【输入样例 2】

```
5
DE
ST
```

DE
ST
WF

【输出样例 2】

37

【数据范围】

对于所有测试数据，保证： $1 \leq n \leq 40$ ；

输入的每个字符串都是合法的，即第一个字符为 F、D、S、M、W 中的一个，第二个字符为 F、W、A、E、T、I、L、K 中的一个。

【示例解释】

示例 1：现有图鉴册中只有一张山脉-火属性的图鉴页，还需要购买其他 39 张。

示例 2：现有图鉴册中有两张沙漠-土属性、两张海洋-雷属性和一张沼泽-火属性的图鉴页（重复的只算一种），还需要购买其他 37 种。

题目解析

题目分析：

本题题意输入 n 个长度为 2 的字符串，并且字符串总共只有 40 种。要求出至少需要添加多少个字符串让这些字符串包含所有 40 种。本题考察字符串以及二维数组/set/map 的运用。

解题思路：

如果需要凑齐 40 种不同的图鉴，就需要知道现在拥有多少种，然后用 40 总数减去现在拥有的个数即可。现在问题就转化为求出所给字符串的种类数，由于输入的 n 个字符串可能会有重复的，因此需要对输入的字符串进行去重处理。对于字符串的去重计数，最简单的是使用 C++ STL 中的 map 映射或者 set 集合，直接将输入的字符串插入到 map 或者 set 中，容器的 size 值就是不重复的字符串个数。当然注意到本题字符串长度只有 2，因此使用二维数组进行桶数组计数也可以解决本题。

参考代码：

```
#include <iostream>
#include <set>
```

```
#include <string>
using namespace std;

int main() {
    int n;
    cin >> n; // 输入 n
    set<string> pages; // 定义 set 集合存储字符串

    for (int i = 0; i < n; i++) {
        string s;
        cin >> s; // 输入字符串
        pages.insert(s); // 将字符串插入集合
    }

    int total = 40;
    int existing = pages.size(); // 集合的大小即为字符串种类数
    int needed = total - existing; // 需要添加的数量就是 40-集合的大小

    cout << needed << endl;

    return 0;
}
```